



시험에 나오는것만 공부한다!

시나공시리즈

기출문제 2026년 1회 정보처리기사 실기



정보처리기사 실기 시험은 한국산업인력공단에서 문제를 공개하지 않아 문제 복원에 많은 어려움이 있습니다. 다음에 제시된 문제는 시험을 치른 학생들의 기억을 토대로 복원한 것이므로, 일부 내용이나 문제별 배점이 실제 시험과 다를 수 있음을 알립니다.

저작권 안내

이 자료는 시나공 카페 회원을 대상으로 하는 자료로서 개인적인 용도로만 사용할 수 있습니다. 허락 없이 복제하거나 다른 매체에 옮겨 실을 수 없으며, 상업적 용도로 사용할 수 없습니다.

*** 수험자 유의사항 ***

1. 시험 문제지를 받는 즉시 응시하고자 하는 종목의 문제지가 맞는지 확인하여야 합니다.
2. 시험 문제지 총면수·문제번호 순서·인쇄상태 등을 확인하고, 수험번호 및 성명을 답안지에 기재하여야 합니다.
3. 문제 및 답안(지), 채점기준은 일절 공개하지 않으며 자신이 작성한 답안, 문제 내용 등을 수험표 등에 이기(옮겨 적는 행위) 등은 관련 법 등에 의거 불이익 조치 될 수 있으니 유의하시기 바랍니다.
4. 수험자 인적사항 및 답안작성(계산식 포함)은 흑색 기구만 사용하되, 동일한 한 가지 색의 필기구만 사용하여 하며 흑색을 제외한 유색 필기구 또는 연필류를 사용하거나 2가지 이상의 색을 혼합 사용하였을 경우 그 문항은 0점 처리됩니다.
5. 답란(답안 기재란)에는 문제와 관련 없는 불필요한 낙서나 특이한 기록사항 등을 기재하여서는 안되며 부정의 목적으로 특이한 표식을 하였다고 판단될 경우에는 모든 문항이 0점 처리됩니다.
6. 답안을 정정할 때에는 반드시 정정부분을 두 줄(=)로 그어 표시하여야 하며, 두 줄로 긋지 않은 답안은 정정하지 않은 것으로 간주합니다.
7. 답안의 한글 또는 영문의 오타자는 오답으로 처리됩니다. 단, 답안에서 영문의 대·소문자 구분, 띄어쓰기는 여부에 관계 없이 채점합니다.
8. 계산 또는 디버깅 등 계산 연습이 필요한 경우는 <문 제> 아래의 연습란을 사용하시기 바라며, 연습란은 채점 대상이 아닙니다.
9. 문제에서 요구한 가지 수(항수) 이상을 답란에 표기한 경우에는 답안기재 순으로 요구한 가지 수(항수)만 채점하고 한 항에 여러 가지를 기재하더라도 한 가지로 보며 그 중 정답과 오답이 함께 기재란에 있을 경우 오답으로 처리됩니다.
10. 한 문제에서 소문제로 파생되는 문제나, 가지수를 요구하는 문제는 대부분의 경우 부분채점을 적용합니다. 그러나 소문제로 파생되는 문제 내에서의 부분 배점은 적용하지 않습니다.
11. 답안은 문제의 마지막에 있는 답란에 작성하여야 합니다.
12. 부정 또는 불공정한 방법(시험문제 내용과 관련된 메모지사용 등)으로 시험을 치른 자는 부정행위자로 처리되어 당해 시험을 중지 또는 무효로 하고, 2년간 국가기술자격검정의 응시자격이 정지됩니다.
13. 시험위원이 시험 중 신분확인을 위하여 신분증과 수험표를 요구할 경우 반드시 제시하여야 합니다.
14. 시험 중에는 통신기기 및 전자기기(휴대용 전화기 등)를 지참하거나 사용할 수 없습니다.
15. 국가기술자격 시험문제는 일부 또는 전부가 저작권법상 보호되는 저작물이고, 저작권자는 한국산업인력공단입니다. 문제의 일부 또는 전부를 무단 복제, 배포, 출판, 전자출판 하는 등 저작권을 침해하는 일체의 행위를 금합니다.

※ 수험자 유의사항 미준수로 인한 채점상의 불이익은 수험자 본인에게 전적으로 책임이 있음

문제 1 디자인 패턴에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. (5점)

- (①) 패턴은 구현부에서 추상층을 분리하여, 서로가 독립적으로 확장할 수 있도록 구성한 패턴으로, 기능과 구현을 두 개의 별도 클래스로 구현한다는 특징이 있다.
- (②) 패턴은 한 객체의 상태가 변화하면 객체에 상속된 다른 객체들에게 변화된 상태를 전달하는 패턴으로, 일대다의 의존성을 정의한다. 주로 분산된 시스템 간에 이벤트를 생성·발행(Publish)하고, 이를 수신(Subscribe)해야 할 때 이용한다.

<보기>

- | | | | |
|-----------|------------------|------------|------------|
| • Builder | • Factory Method | • Adapter | • Bridge |
| • Facade | • Proxy | • Observer | • Mediator |

답

- ①
- ②

문제 2 비기능 요구사항에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 답을 <보기>에서 찾아 쓰시오. (5점)

- (①) : Windows, macOS, Linux 등 다양한 운영체제에서 정상적으로 동작해야 한다.
- (②) : 시스템은 메모리 사용량을 최소화하여 제한된 자원 환경에서도 안정적으로 실행되어야 한다.
- (③) : 사용자의 요청에 대해 1초 이내에 응답해야 한다.

<보기>

- | | | |
|-----------|-----------|----------|
| • 운영요구사항 | • 자원요구사항 | • 성능요구사항 |
| • 시스템요구사항 | • 사용자요구사항 | |

답

- ①
- ②
- ③

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 3 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) (5점)

```
#include <stdio.h>
double arr1(int p[], int len) {
    double av = 0;
    int i;
    for(i = 0; i < len; i++) {
        av += (double)p[i];
    }
    return av / len;
}
double arr2(int *p, int len) {
    double av = 0;
    int i;
    for(i = 0; i < len; i++) {
        av += (double)*(p + i);
    }
    return av / len;
}
int main( ) {
    int arr[10] = {70, 50, 80, 30, 45, 55, 65, 95, 15, 25};
    int len = 10;
    printf("%.2f", arr1(arr, len) + arr2(arr, len));
    return 0;
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 4 다음은 CREATE 문을 이용해 <PLAYER> 테이블을 생성하는 <SQL> 문이다. <TEAM> 테이블의 TEAM_ID2 컬럼을 참조하여 외래키를 설정하려고 한다. 괄호(①~⑤)에 알맞은 구문을 넣어 <SQL문>을 완성하시오. (단, 외래키 제약조건의 명칭은 TEAM_TF로 정의한다.) (5점)

<SQL문>

```
CREATE TABLE PLAYER (  
    PLAYER_ID CHAR(7) NOT NULL,  
    PLAYER_NAME VARCHAR2(20) NOT NULL,  
    TEAM_ID CHAR(3) NOT NULL,  
    PRIMARY KEY (PLAYER_ID),  
    ( ① ) TEAM_TF ( ② ) KEY ( ③ ) ( ④ ) TEAM ( ⑤ )
```

답

- ①
- ②
- ③
- ④
- ⑤

문제 5 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
lst = list(range(10))  
for c in lst[::-2]:  
    print(c, end='A')  
print( )
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 6 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
class A {
    String f(Object x) {
        return "1";
    }
    String g( ) {
        return f("a");
    }
}
class B extends A {
    String f(Object x) {
        return "2";
    }
    String f(String x) {
        return "3";
    }
}
class Main {
public static void main(String[] args) {
    A a = new B( );
    System.out.println(a.g( ));
}
}
```

답 :

문제 7 정보 보호에 대한 다음 설명에 해당하는 용어를 영문 약어로 쓰시오. (5점)

정보 자산을 안전하게 보호하기 위한 보호 절차와 대책으로, 정보보호 관리 체계라고 한다. 조직에 맞는 정보보호 정책을 수립하고, 위험에 상시 대응하는 여러 보안 대책을 통합 관리한다. 공공 부문과 민간 기업 부문에서 이것을 평가하고 인증하는 사업을 한국인터넷진흥원(KISA)에서 운영중이다.

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 8 A의 IP 주소는 192.168.11.20이고, B의 IP 주소는 192.168.12.200이다. 서브넷 마스크가 255.255.254.0일 때, A와 B가 속한 네트워크의 네트워크 주소를 CIDR 표기법으로 작성하시오. (5점)

답

- ① A의 네트워크 주소 :
- ② B의 네트워크 주소 :

문제 9 보안 위협에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. (5점)

목표 조직이 자주 방문하는 웹 사이트를 사전에 감염시켜 목표 조직의 일원이 웹 사이트에 방문했을 때 악성 코드에 감염되게 한다. 이후에는 감염된 PC를 기반으로 조직의 중요 시스템에 접근하거나 불능으로 만드는 등의 영향력을 행사하는 웹 기반 공격이다.

<보기>

- | | | | |
|------------|---------------------|-----------------|-----------------|
| • Pharming | • Drive-by Download | • Watering Hole | • Business SCAM |
| • Phishing | • Cyber Kill Chain | • Ransomware | • Sniffing |

답 :

문제 10 다음은 데이터베이스 구축까지의 과정을 나열한 것이다. 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. (5점)

(①) → (②) → (③) → (④) → (⑤)

<보기>

- | | | | |
|------------|-------------|----------|------------|
| • 개념적 설계 | • 데이터베이스 구현 | • 물리적 설계 | • 요구 조건 분석 |
| • 인터페이스 설계 | • 논리 스키마 설계 | • 논리적 설계 | • 트랜잭션 작성 |

답

- ①
- ②
- ③
- ④
- ⑤

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 11 다음 <EMPLOYEE>와 <DEPT> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오.
(5점)

<EMPLOYEE>			<DEPT>		
emp_id	emp_name	dep_id	dep_id	dep_name	budget
E01	김은소	D01	D01	총무부	3,000
E02	강동준	D02	D02	인사부	5,000
E03	고희식	D03	D03	개발부	7,000
E04	황진주	D04	D04	영업부	9,000

<SQL문>

```
SELECT COUNT(*) FROM EMPLOYEE e JOIN DEPT d
ON e.dep_id = d.dep_id
WHERE d.budget > (SELECT AVG(budget) FROM DEPT)
```

답 :

문제 12 모듈에 대한 다음 설명에서 각 지문(①~③)에 해당하는 응집도(Cohesion)를 <보기>에서 찾아 기호 (㉠~㉣)로 쓰시오. (5점)

- ① 내부의 요소들이 기능적으로 연관성은 없으나, 순차적으로 실행될 때의 응집도
- ② 서로 다른 기능을 수행하지만 동일한 입력과 출력을 사용할 때의 응집도
- ③ 하나의 기능에 밀접하게 관련되어 있거나 연관되어 있을 때의 응집도

<보기>

- ㉠ 기능적 응집도 ㉡ 순차적 응집도 ㉢ 교환적 응집도 ㉣ 절차적 응집도
- ㉤ 시간적 응집도 ㉥ 논리적 응집도 ㉦ 우연적 응집도

답

- ㉠
- ㉡
- ㉢

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 13 애플리케이션 테스트에 관한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. (5점)

- 하향식 통합 테스트는 프로그램의 상위 모듈에서 하위 모듈 방향으로 통합하면서 테스트하는 기법이다. 깊이 우선 통합법이나 넓이 우선 통합법을 사용하며, 주요 제어 모듈의 종속 모듈들을 (①)으로 대체한다는 특징이 있다.
- 상향식 통합 테스트는 프로그램의 하위 모듈에서 상위 모듈 방향으로 통합하면서 테스트하는 기법이다. 하위 모듈들을 클러스터(Cluster)로 결합하며, 상위 모듈에서 데이터의 입·출력을 확인하기 위해 더미 모듈인 (②)를 작성한다는 특징이 있다.

답

- ①
- ②

문제 14 다음 설명에서 괄호에 공통으로 들어갈 알맞은 답을 쓰시오. (5점)

()는 파일이나 디렉터리를 가리키는 링크 파일이다. Windows의 바로 가기 아이콘과 비슷한 개념으로, 다른 위치에 있는 파일이나 디렉터리의 경로 정보를 저장하여 해당 대상에 접근할 수 있게 한다.
레이스 컨디션 공격의 한 종류인 () 공격은 root 권한을 가진 프로세스가 임시 파일을 생성하는 과정에서 발생한다. 공격자는 임시 파일이 생성된 직후 이를 삭제하고, 같은 이름의 ()를 만들어 중요한 파일을 가리키도록 설정한다.
이를 방지하기 위해 프로그램은 임시 파일을 사용하기 전에 해당 파일이 ()인지 확인하고, ()가 아닌 경우에만 사용하거나 필요 시 삭제한다.

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 15 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
#include <stdio.h>
struct fns {
    int* (*fn)(int*);
};

int* dummy(int *d) {
    return d + 1;
}

int main( ) {
    struct fns mine;
    int n[] = {16, 32};
    mine.fn = dummy;
    printf("%x", *mine.fn(n));
    return 0;
}
```

답 :

문제 16 다음 Python으로 구현된 프로그램을 분석하여 HumanDev를 입력했을 때 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
i = input( )
x = []
for word in i.split( ):
    x.append(word)
y = ' '.join(x)
z = ".join(c for c in y[::-1] if c not in 'ong')
print(z)
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 17 HDLC(High-level Data Link Control)에 대한 다음 설명에서 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. (5점)

HDLC는 비트(Bit) 위주의 프로토콜로, 각 프레임에 데이터 흐름을 제어하고 오류를 검출할 수 있는 비트 열을 삽입하여 전송한다. 포인트 투 포인트(Point-to-Point) 및 멀티 포인트(Multi-Point), 루프(Loop) 등 다양한 데이터 링크 형태에 동일하게 적용이 가능하다는 특징이 있다.

HDLC의 프레임 구조는 헤더, 텍스트, 트레일러로 구분되며, 헤더는 다시 플래그, 주소부, 제어부로 구분할 수 있는데, 제어부에는 프레임의 종류를 식별하기 위해 사용한다. 제어부의 첫 번째, 두 번째 비트를 사용하여 (①) 프레임, (②) 프레임, (③) 프레임으로 구분한다.

(①) 프레임은 I 프레임으로 불리며, 제어부가 '0'으로 시작하는 프레임으로, 사용자 데이터를 전달하거나 피기백킹(Piggybacking) 기법을 통해 데이터에 대한 확인 응답을 보낼 때 사용된다.

(②) 프레임은 S 프레임으로 불리며, 제어부가 '10'으로 시작하는 프레임으로, 오류 제어와 흐름 제어를 위해 사용된다.

(③) 프레임은 U 프레임으로 불리며, 제어부가 '11'로 시작하는 프레임으로, 링크의 동작 모드 설정과 관리를 한다.

(③) 프레임에서 설정할 수 있는 동작 모드에는 표준 응답 모드, (④), (⑤)의 세 가지로 구분된다.

표준 응답 모드는 반이중 통신을 하는 포인트 투 포인트(Point-to-Point) 또는 멀티 포인트(Multi-Point) 불균형 링크 구성에 사용되며, 종국은 주국의 허가(Poll)가 있을 때에만 송신하는 특징이 있다.

(④)는 포인트 투 포인트(Point-to-Point) 균형 링크에서 사용되며, 혼합국끼리 허가 없이 언제나 전송할 수 있다.

(⑤)는 전이중 통신을 하는 포인트 투 포인트(Point-to-Point) 불균형 링크 구성에 사용되며, 종국은 주국의 허가(Poll) 없이도 송신이 가능하지만 링크 설정이나 오류 복구 등의 제어 기능은 주국만 가능하다.

<보기>

• 비동기 응답 모드	• 주소부	• 제어부	• ARQ	• 정보
• 비번호	• 감독	• 플래그	• 비동기 균형 모드	

답

- ①
- ②
- ③
- ④
- ⑤

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 18 학생(STUDENT) 테이블에 전기과 학생이 50명, 전산과 학생이 100명, 전자과 학생이 50명 있다고 할 때, 다음 SQL문 ①, ②, ③의 실행 결과로 표시되는 튜플의 수를 쓰시오. (단, DEPT 필드는 학과를 의미한다) (5점)

- ① SELECT DEPT FROM STUDENT;
② SELECT DISTINCT DEPT FROM STUDENT;
③ SELECT COUNT(DISTINCT DEPT) FROM STUDENT WHERE DEPT = '전산과';

답

- ①
- ②
- ③

문제 19 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
class Calc {
    int cc(int a, int b) {
        return a + b;
    }
    int cc(char a, char b) {
        return a - b;
    }
    char cc(char c) {
        return c;
    }
}

class Main {
public static void main(String[] args) {
    Calc calc = new Calc( );
    int a = calc.cc(5, 4);
    int b = calc.cc('c', 'a');
    char c = calc.cc('3');
    System.out.println(a + b + "2" + c);
}
}
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하시기 바랍니다.

문제 20 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) (5점)

```
def f(a):
    m = [[x] for x in a]
    b = m[:]
    for i in range(len(b) - 1):
        b[i+1] += b[i]
    return sum(len(x) for x in m)
print(f([1, 2, 3, 4]))
```

답 :

연 습 란

※ 다음 여백은 연습란으로 사용하기 바랍니다.

기출문제 정답 및 해설

[문제 1]

① Bridge ② Observer

[문제 2]

① 운영요구사항 ② 자원요구사항 ③ 성능요구사항

[문제 3]

106.00

※ **답안 작성 시 주의 사항** : 프로그램의 실행 결과는 부분 점수가 없으므로 정확하게 작성해야 합니다. 예를 들어, 소수점을 생략하거나 자릿수를 잘못 지정하여 106 또는 106.0으로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 처리됩니다.

[해설]

```
#include <stdio.h>

④ double arr1(int p[], int len) {
⑤     double av = 0;
⑥     int i;
⑦     for(i = 0; i < len; i++) {
⑧         av += (double)p[i];
⑨     }
⑩     return av / len;
⑪ }

⑫ double arr2(int *p, int len) {
⑬     double av = 0;
⑭     int i;
⑮     for(i = 0; i < len; i++) {
⑯         av += (double)*(p + i);
⑰     }
⑱     return av / len;
⑲ }

int main( ) {
①     int arr[10] = {70, 50, 80, 30, 45, 55, 65, 95, 15, 25};
②     int len = 10;
③⑩⑰ printf("%.2f", arr1(arr, len) + arr2(arr, len));
⑱     return 0;
}
```

모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

- ① 10개의 요소를 갖는 정수형 배열 arr을 선언하고 초기화한다. 정수형 데이터는 요소당 4Byte씩 공간을 차지하므로, 메모리 주소는 4씩 증가하며 연속된 공간에 할당된다. (여기서 지정한 주소는 이해를 돕기 위해 임의로 설정한 10진수 값입니다.)

메모리

- ⑫ 실수형 변수 av를 선언하고 0으로 초기화한다.
- ⑬ 정수형 변수 i를 선언한다.
- ⑭ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 i가 len보다 작은 동안 ⑮번을 반복 수행한다. len이 10이므로 ⑮번을 총 10회 반복 수행한다.
- ⑮ 'av = av + (double)((p+i))'와 동일하다. (p+i)가 가리키는 값을 실수형으로 변환하여 av에 누적한다.
- ※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	*(p+i)	av
		0
0	70	70
1	50	120
2	80	200
3	30	230
4	45	275
5	55	330
6	65	395
7	95	490
8	15	505
9	25	530

- ⑯ 'av / len', 즉 '530/10'의 결과인 53을 함수를 호출했던 ⑰번으로 반환한다.
- ⑰ ⑨번에서 돌려받은 값 53과 ⑯번에서 돌려받은 값 53의 합인 106을 "%.2f" 서식에 맞춰 정수 부분은 모두 출력하고 소수점 이하는 3자리에서 반올림하여 2자리까지만 표시한 106.00을 출력한다.
- 결과 106.00

[문제 4]

- ① CONSTRAINT ② FOREIGN ③ TEAM_ID ④ REFERENCES ⑤ TEAM_ID2

[해설]

<pre>CREATE TABLE PLAYER (PLAYER_ID CHAR(7) NOT NULL, PLAYER_NAME VARCHAR2(20) NOT NULL, TEAM_ID CHAR(3) NOT NULL, PRIMARY KEY (PLAYER_ID), CONSTRAINT TEAM_TF FOREIGN KEY (TEAM_ID) REFERENCES TEAM (TEAM_ID2));</pre>	<PLAYER> 테이블을 생성한다. 'PLAYER_ID' 속성은 문자 7자로 NULL 값을 갖지 않는다. 'PLAYER_NAME' 속성은 가변길이 문자 20자로 NULL 값을 갖지 않는다. 'TEAM_ID' 속성은 문자 3자로 NULL 값을 갖지 않는다. 'PLAYER_ID'를 기본키로 정의한다. 외래키 제약 조건의 이름은 'TEAM_TF'이며, 'TEAM_ID' 속성을 <TEAM> 테이블의 'TEAM_ID2' 속성을 참조하는 외래키로 정의한다.
---	--

[문제 5]

9A7A5A3A1A

- ※ **답안 작성 시 주의 사항** : 프로그램의 실행 결과는 부분 점수가 없으므로 정확하게 작성해야 합니다. 예를 들어, 대소문자 구분 없이 9a7a5a3a1a 또는 9A7a5A3a1A로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 처리됩니다.

[해설]

```
① lst = list(range(10))
② for c in lst[::-2]:
③     print(c, end='A')
④ print( )
```

- ① 리스트 lst를 선언하고, 0부터 9(10-1)까지의 정수로 초기화한다.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
lst	0	1	2	3	4	5	6	7	8	9
	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

❷ 반복 변수 c에 리스트 lst의 0번째 위치부터 마지막 위치까지 -2씩 증가하면서 해당 위치의 요소들을 순차적으로 저장하며 ❸번을 반복 수행한다.

※ 객체명[::증가값] : 객체의 0번째 위치에서 마지막 위치까지 '증가값'만큼 증가하면서 해당 위치의 요소들을 가져옴

※ 증가값이 음수일 경우 뒤에서 앞으로 이동하는데, 초기위치가 생략되었으므로 맨 뒤인 -1이 초기위치가 되며, 앞으로 2칸씩 이동함

❸ c의 값을 출력하고 종료문자로 'A'를 출력한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

c	출력
9	9A
7	9A7A
5	9A7A5A
3	9A7A5A3A
1	9A7A5A3A1A

❹ 인수가 없으므로 커서의 위치를 다음 줄 처음으로 이동한다. 즉 줄바꿈만 수행한다.

결과 9A7A5A3A1A

[문제 6]

2

[해설]

```
class A {
    ❸ String g( ) {
    ④⑦ return f("a");
    }
    String f(Object x) {
        return "1";
    }
}
class B extends A {
    ⑤ String f(Object x) {
    ⑥ return "2";
    }
    String f(String x) {
        return "3";
    }
}
class Main {
    public static void main(String[] args) {
    ① A a = new B( );
    ②⑧ System.out.println(a.g( ));
    }
}
```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

❶ B 클래스의 생성자를 이용하여 A 클래스형 객체 변수 a를 선언한다.

- [부모클래스명] [객체변수명] = new [자식클래스생성자()] : 부모 클래스의 객체 변수를 선언하면서 자식 클래스의 생성자를 사용하면 형 변환이 발생한다.
 - 이렇게 형 변환이 발생했을 때 부모 클래스와 자식 클래스에 동일한 메소드가 있으면 자식 클래스의 메소드로 재정의된다.
- 2 a.g() 메소드를 호출한 후 반환받은 값을 출력한다. 먼저 a.g()를 호출한다. 3번으로 이동한다.
 - 3 문자열을 반환하는 g() 메소드의 시작점이다.
 - 4 “a”를 인수로 f() 메소드를 호출한 후 반환받은 값을 메소드를 호출했던 곳으로 반환한다. 먼저 f(“a”)를 호출하기 위해 5번으로 이동한다.
- ※ 현재 g() 메소드는 A 클래스에 정의되어 있으므로 A 클래스의 f() 메소드를 찾습니다. A 클래스에는 이와 관련된 String f(Object x) 메소드가 있으므로, f(“a”)는 f(Object x)를 호출합니다. 하지만 1번에서 형 변환이 발생했으므로 실행 시에는 자식 클래스 B의 f(Object x) 메소드가 수행됩니다.
- 5 문자열을 반환하는 f(Object x) 메소드의 시작점이다. 4번에서 전달한 “a”를 매개변수 x가 받는다.
 - 6 문자열 2를 메소드를 호출했던 7번으로 반환한다.
 - 7 6번에서 돌려받은 값 2를 메소드를 호출했던 8번으로 반환한다.
 - 8 7번에서 돌려받은 값 2를 출력한 후 커서를 다음 줄 처음으로 이동한다.

결과 2

[문제 7]

ISMS

[문제 8]

① 192.168.10.0/23 ② 192.168.12.0/23

[해설]

- CIDR(Classless Inter-Domain Routing)은 IP 주소 뒤에 /비트수를 붙여 네트워크 부분의 길이를 표시하는 방법으로, 서브넷 마스크의 1의 비트수로 표현합니다.
 - 서브넷 마스크 255.255.254.0을 2진수로 변환한 후 1의 개수를 구합니다.

11111111.11111111.11111110.00000000
 - 1의 개수가 23이므로 /비트수는 /23입니다.
- A의 네트워크 주소는 A의 IP 주소와 서브넷 마스크를 2진수로 변환한 후 AND 연산하여 구합니다.

A의 IP 주소	11000000.10101000.00001011.00010100	← 192.168.11.20
서브넷 마스크	11111111.11111111.11111110.00000000	← 255.255.254.0
AND 연산	11000000.10101000.00001010.00000000	← 192.168.10.0

∴ A의 네트워크 주소를 CIDR 표기법으로 표현하면 192.168.10.0/23입니다.
- B의 네트워크 주소는 B의 IP 주소와 서브넷 마스크를 2진수로 변환한 후 AND 연산하여 구합니다.

B의 IP 주소	11000000.10101000.00001100.11001000	← 192.168.12.200
서브넷 마스크	11111111.11111111.11111110.00000000	← 255.255.254.0
AND 연산	11000000.10101000.00001100.00000000	← 192.168.12.0

∴ B의 네트워크 주소를 CIDR 표기법으로 표현하면 192.168.12.0/23입니다.

[문제 9]

Watering Hole

[문제 10]

① 요구 조건 분석 ② 개념적 설계 ③ 논리적 설계 ④ 물리적 설계 ⑤ 데이터베이스 구현

[문제 11]

2

[해설]

문제의 질의문은 하위 질의가 있는 질의문입니다. 다음과 같이 안쪽 하위 질의의 SELECT문을 먼저 검색한 다음 그 검색 결과를 본 질의의 조건에 적용하면 됩니다.

```
SELECT COUNT(*)
FROM EMPLOYEE e JOIN DEPT d ON e.dep_id = d.dep_id
WHERE d.budget > (SELECT AVG(budget) FROM DEPT)
```

❶

<하위 질의 ❶>

SELECT AVG(budget)	'budget' 필드의 평균을 표시한다.
FROM DEPT	<DEPT> 테이블을 대상으로 조회한다.

<결과>

평균
6000

<본 질의>

SELECT COUNT(*)	조회된 레코드의 개수를 표시한다.
FROM EMPLOYEE e JOIN DEPT d	• <EMPLOYEE> 테이블과 <DEPT> 테이블을 대상으로 조회한다. • <EMPLOYEE>, <DEPT> 테이블의 별칭으로 <e>, <d>를 지정한다.
ON e.dep_id = d.dep_id	<e>는 <EMPLOYEE> 테이블을, <d>는 <DEPT> 테이블을 의미한다.
WHERE d.budget > (❶)	<EMPLOYEE> 테이블의 'dep_id'와 <DEPT> 테이블의 'dep_id'를 기준으로 JOIN 한다. <DEPT> 테이블의 'budget'이 6000을 초과하는 자료만을 대상으로 한다.

<결과>

<EMPLOYEE>			<DEPT>		
emp_id	emp_name	dep_id	dep_id	dep_name	budget
E01	김은소	D01	D01	총무부	3,000
E02	강동준	D02	D02	인사부	5,000
E03	고희식	D03	D03	개발부	7,000
E04	황진주	D04	D04	영업부	9,000

∴ 질의의 결과로 표시되는 레코드의 개수는 2입니다.

[문제 12]

① ㉠ ② ㉡ ③ ㉢

[문제 13]

※ 각 문항별로 다음 중 하나를 쓰면 됩니다.

- ① 스텝, Stub
 ② 드라이버, Driver

[문제 14]

※ 다음 중 하나를 쓰면 됩니다.

심볼릭 링크, Symbolic Link

[문제 15]

20

※ **답안 작성 시 주의 사항** : 프로그램의 실행 결과는 부분 점수가 없으므로 정확하게 작성해야 합니다. 예를 들어, 출력값을 16진수가 아니라 10진수인 32로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 처리됩니다.

[해설]

```
#include <stdio.h>
struct fns {
    int* (*fn)(int*);
};

❶ int* dummy(int *d) {
❷     return d + 1;
❸ }

int main( ) {
❹     struct fns mine;
❺     int n[] = {16, 32};
❻     mine.fn = dummy;
❼❽ printf("%x", *mine.fn(n));
❾     return 0;
}
```

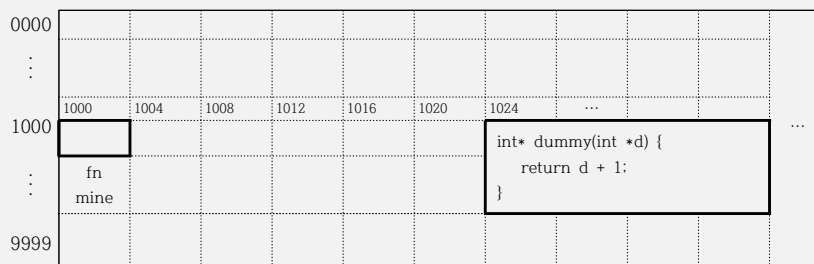
모든 C 언어 프로그램은 반드시 main() 함수에서 시작한다.

❶ fns 구조체 형태로 객체 변수 mine을 선언한다. mine은 함수 포인터 fn 하나를 저장하기 위한 메모리 공간이 확보된다.

※ 포인터 변수는 32비트 시스템 환경을 기준으로 4Byte로 가정하였으며, 여기서 지정한 주소는 이해를 돕기 위해 임의로 설정한 10진수 값입니다.

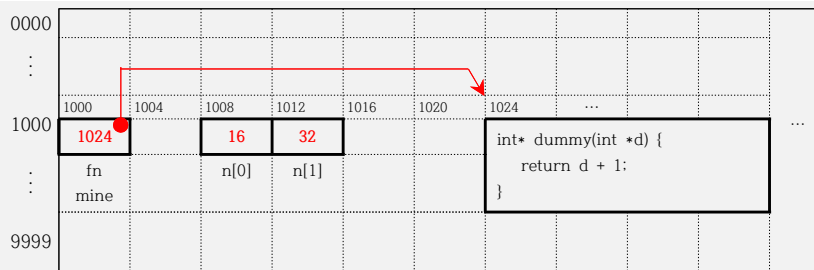
※ 코드 실행 전 번역 과정에서 코드는 함수 단위로 메모리에 저장되는데, 여기서는 설명을 위해 dummy 함수가 저장된 형태를 표현했으며, 그 시작 주소는 임의로 지정한 것입니다.

메모리



❷ 2개의 요소를 갖는 정수형 배열 n을 선언하고 초기화한다.

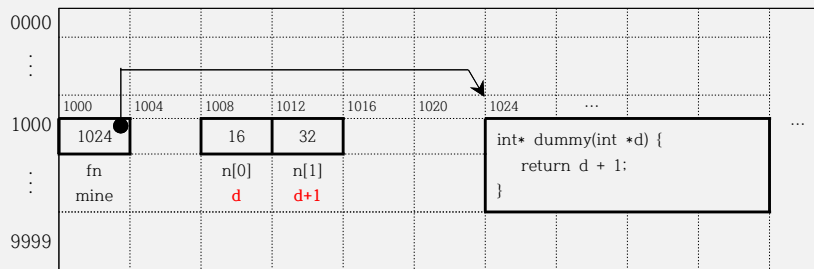
❸ 구조체 변수 mine의 멤버인 함수 포인터 fn에 dummy 함수의 시작 주소를 저장한다.



④ n을 인수로 mine의 fn, 즉 dummy 함수를 호출한 후 반환받은 값을 출력한다.

※ 인수로 배열의 이름을 지정하면 배열의 시작 주소가 인수로 전달됩니다.

⑤ 반환형이 정수형 포인터(int*)인 dummy 함수의 시작점이다. ④번에서 전달받은 n 배열의 주소 1008을 정수형 포인터 변수 d가 받는다.



⑥ 'd+1' 즉 포인터 주소에 1을 더한다는 것은 자료형의 크기만큼 주소 값이 증가한다는 것이다. 여기서 포인터 변수의 자료형은 4Byte로 가정했으므로, d에 저장된 주소 1008에서 4Byte 증가한 1012를 함수를 호출했던 ⑦번으로 반환한다.

⑦ ⑥번에서 돌려받은 1012가 가리키는 값 32를 16진수 형식인 %x 서식으로 출력하므로 20이 출력된다.

※ $32_{10} = 32 / 16 = 2(\text{몫}), 0(\text{나머지}) = 20_{16}$

※ *mine.fn(n), 즉 *(1012)는 1012 주소가 저장하고 있는 값을 의미합니다.

결과 20

[문제 16]

veDamuH

※ **답안 작성 시 주의 사항** : 프로그램의 실행 결과는 부분 점수가 없으므로 정확하게 작성해야 합니다. 예를 들어, 출력값을 대소문자 구분 없이 vedamuh나 VEDAMUH로 썼을 경우 부분 점수 없이 완전히 틀린 것으로 처리됩니다.

[해설]

```

① i = input( )
② x = []
③ for word in i.split( ):
④     x.append(word)
⑤ y = ' '.join(x)
⑥ z = ''.join(c for c in y[::-1] if c not in 'ong')
⑦ print(z)

```

① 변수 i를 선언하고 키보드로 입력받은 문자열로 초기화한다. 문제에서 HumanDev를 입력받았다고 가정하였으므로 i에는 HumanDev로 초기화된다.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
i	H	u	m	a	n	D	e	v

② 빈 리스트 x를 선언한다.

③ 반복 변수 word에 변수 i를 공백을 기준으로 분리하여 리스트의 형태로 반환하면서, 리스트의 요소 수만큼

④번을 반복 수행한다.

- **split(값)** : ‘값’을 기준으로 문자열을 분리하여 리스트로 반환하며, ‘값’을 생략하면 공백으로 문자열을 분리함
- i에 저장된 값에 공백이 없으므로 단일 요소를 가진 ["HumanDev"] 리스트가 반환됩니다.
- 반환된 리스트의 요소가 1개이므로 ④번은 1회만 수행됩니다.

④ word의 값을 리스트 x의 맨 뒤에 추가한다.

x ["HumanDev"]

⑤ 리스트 x 내부의 문자열 요소들을 공백(' ') 한 칸을 사이에 두고 하나의 문자열로 결합하여 변수 y에 저장한다.

y "HumanDev"

- **구분자.join()** : 문자열들을 지정한 구분자로 연결하여 하나의 문자열로 반환함
- 리스트 x 내의 요소가 "HumanDev" 하나뿐이므로 결합할 사이 공백 없이 문자열 "HumanDev"가 그대로 변수 y에 저장됩니다.

⑥ 변수 y에 저장된 문자열을 뒤에서부터 하나씩 추출하면서 해당 문자가 'o', 'n', 'g'가 아닌 경우만 결합하여 변수 z에 저장한다. z에는 "veDamuH"가 저장된다.

z "veDamuH"

z = ''.join(c for c in y[::-1] if c not in 'ong')

①

②

- ① : 제너레이터 표현식(Generator Expression) 구문으로, y를 역순으로 순회하면서 if 문의 조건이 참인 경우 c를 생성하여 반환합니다. 즉 'v', 'e', 'D', 'a', 'm', 'u', 'H'가 순차적으로 반환됩니다.
- ② ''.join(c) : 공백없이 ①에서 반환되는 문자들을 연결하여 하나의 문자열로 반환합니다.
- ∴ y에 저장된 "HumanDev"를 역순으로 순회하면서 'n'을 제외하므로 "veDamuH"가 z에 저장됩니다.
- ※ **객체명[::-증가값]** : 객체의 0번째 위치에서 마지막 위치까지 '증가값'만큼 증가하면서 해당 위치의 요소들을 가져옴
- ※ 증가값이 음수일 경우 뒤에서 앞으로 이동하는데, 초기위치가 생략되었으므로 맨 뒤인 -1이 초기위치가 됩니다.
- ※ **제너레이터 표현식(Generator Expression)** : (표현식 for 변수 in 반복가능객체 [if 조건]) 형태로 작성하며, 표현식의 결과를 필요할 때마다 하나씩 생성하는 객체(Generator)를 만들

⑦ z의 값 veDamuH를 출력한다.

결과 veDamuH

[문제 17]

① 정보 ② 감독 ③ 비번호 ④ 비동기 균형 모드 ⑤ 비동기 응답 모드

[문제 18]

① 200 ② 3 ③ 1

[풀이]

①

SELECT DEPT	'DEPT'를 표시한다.
FROM STUDENT;	<STUDENT> 테이블을 대상으로 검색한다.

- <STUDENT> 테이블에서 'DEPT'를 검색합니다. 총 200개의 튜플이 들어 있고 검색 조건이 없으므로 튜플의 수는 200입니다.

②

SELECT DISTINCT DEPT	'DEPT'를 표시하되, 같은 'DEPT' 속성의 값은 한 번만 표시한다.
FROM STUDENT;	<STUDENT> 테이블을 대상으로 검색한다.

- <STUDENT> 테이블에서 'DEPT'를 검색하는 데 중복된 결과는 처음의 한 개만 검색에 포함시킵니다. 전기과 50개 튜플의 'DEPT' 속성의 값이 같으므로 1개, 전산과 100개 튜플의 'DEPT' 속성의 값이 같으므로 1개, 전자과 50개 튜플의 'DEPT' 속성의 값이 같으므로 1개를 검색에 포함시키므로 3개의 튜플이 검색됩니다.

③

SELECT COUNT(DISTINCT DEPT)	'DEPT'의 개수를 표시하되, 같은 'DEPT' 속성의 값은 한 번만 계산한다.
FROM STUDENT;	<STUDENT> 테이블을 대상으로 검색한다.
WHERE DEPT = '전산과';	'DEPT'가 "전산과"인 자료만을 대상으로 검색한다.

- <STUDENT> 테이블에서 'DEPT' 속성의 값이 '전산과'인 튜플에 대해 중복을 제거하고 개수를 세므로 1이 검색 결과로 표시됩니다.

[문제 19]

1123

[해설]

```
class Calc {
  ① ③ int cc(int a, int b) {
    ④    return a + b;
  }
  ② ⑥ int cc(char a, char b) {
    ⑦    return a - b;
  }
  ③ ⑨ char cc(char c) {
    ⑩    return c;
  }
}

class Main {
  public static void main(String[] args) {
    ①    Calc calc = new Calc( );
    ②    int a = calc.cc(5, 4);
    ⑤    int b = calc.cc('c', 'a');
    ③    char c = calc.cc('3');
    ⑩    System.out.println(a + b + "2" + c);
  }
}
```

- ① 정수를 반환하는 cc(int a, int b) 메소드를 정의한다. 인수가 int형 2개일 때 호출된다.
- ② 정수를 반환하는 cc(char a, char b) 메소드를 정의한다. 인수가 char형 2개일 때 호출된다.
- ③ 문자를 반환하는 cc(char c) 메소드를 정의한다. 인수가 char형 1개일 때 호출된다.
- ※ ①, ②, ③ 세 개의 메소드는 이름은 모두 cc로 같지만 매개변수의 개수 또는 자료형이 다르므로 서로 다른 메소드입니다.
- ※ 이처럼 하나의 클래스 내에서 메소드 이름은 같게 하고 매개변수의 개수나 자료형을 다르게 하여 여러 메소드를 정의하는 것을 오버로딩(Overloading)이라고 합니다.

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- ❶ Calc 클래스의 객체 변수 calc를 선언하고 Calc 클래스의 객체를 생성한다.
- ❷ 정수형 변수 a를 선언하고, 5와 4를 인수로 calc의 cc() 메소드를 호출한 후 돌려받은 값으로 초기화한다. 정수형 변수 2개가 인수로 전달되므로, ㉠의 cc(int a, int b) 메소드가 호출된다. ❸번으로 이동한다.
- ❸ 정수를 반환하는 cc() 메소드의 시작점이다. ❷번에서 전달한 5와 4를 각각 a와 b가 받는다.
- ❹ a + b의 결과인 9를 메소드를 호출했던 ❺번으로 반환한다.
- ❺ 정수형 변수 b를 선언하고, 'c'와 'a'를 인수로 calc의 cc() 메소드를 호출한 후 돌려받은 값으로 초기화한다. 문자형 변수 2개가 인수로 전달되므로, ㉡의 cc(char a, char b) 메소드가 호출된다. ❻번으로 이동한다.
- ❻ 정수를 반환하는 cc() 메소드의 시작점이다. ❺번에서 전달한 'c'와 'a'를 각각 a와 b가 받는다.
 - ※ 문자는 내부적으로 문자(ASCII) 코드값으로 변환되어 'c'는 99, 'a'는 97로 처리됩니다.
 - 알파벳 소문자의 아스키코드 값은 'a(97)' ~ 'z(122)'입니다.
 - 알파벳 대문자의 아스키코드 값은 'A(65)' ~ 'Z(90)'입니다.
- ❼ a - b의 결과인 2를 메소드를 호출했던 ❸번으로 반환한다.
- ❽ 문자형 변수 c를 선언하고, '3'을 인수로 calc의 cc() 메소드를 호출한 후 돌려받은 값으로 초기화한다. 문자형 변수 1개가 인수로 전달되므로, ㉢의 cc(char c) 메소드가 호출된다. ❹번으로 이동한다.
- ❹ 문자를 반환하는 cc() 메소드의 시작점이다. ❸번에서 전달한 '3'을 매개변수 c가 받는다.
- ❿ 매개변수 c의 값 '3'을 메소드를 호출했던 ❸번으로 반환한다.
- ⓫ a + b + "2" + c의 결과인 1123을 출력한 후 커서를 다음 줄 처음으로 이동한다.

a + b + "2" + c

㉠

㉡

㉢

- ㉠ a + b : '숫자+숫자'이므로 9+2 = 11
- ㉡ ㉠ + "2" : '숫자+문자열'이므로 11+"2" = "112"
- ㉢ ㉡ + c : '문자열+문자'이므로 "112"+"3" = "1123"

※ '숫자+숫자'는 두 숫자를 합한 값을 출력하지만, '문자열+숫자', '숫자+문자열', '문자열+문자'와 같이 문자와 숫자가 섞인 경우에는 모두 문자열로 인식되므로 값이 붙어서 출력됩니다.

결과 1123

[문제 20]

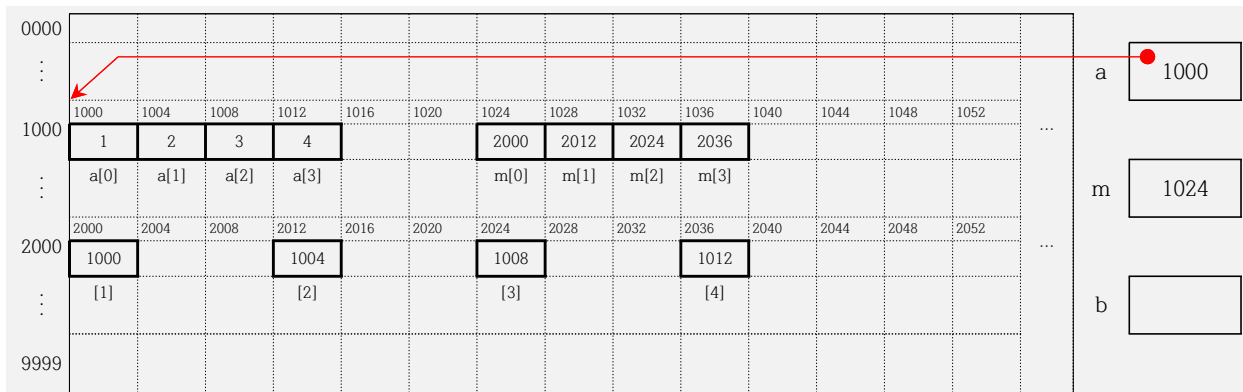
10

[해설]

```
❷ def f(a):                                # def는 Python에서 메소드를 정의하는 예약어이다.
❸     m = [[x] for x in a]
❹     b = m[:]
❺     for i in range(len(b) - 1):
❻         b[i+1] += b[i]
❼         return sum(len(x) for x in m)
❿❸ print(f([1, 2, 3, 4]))
```

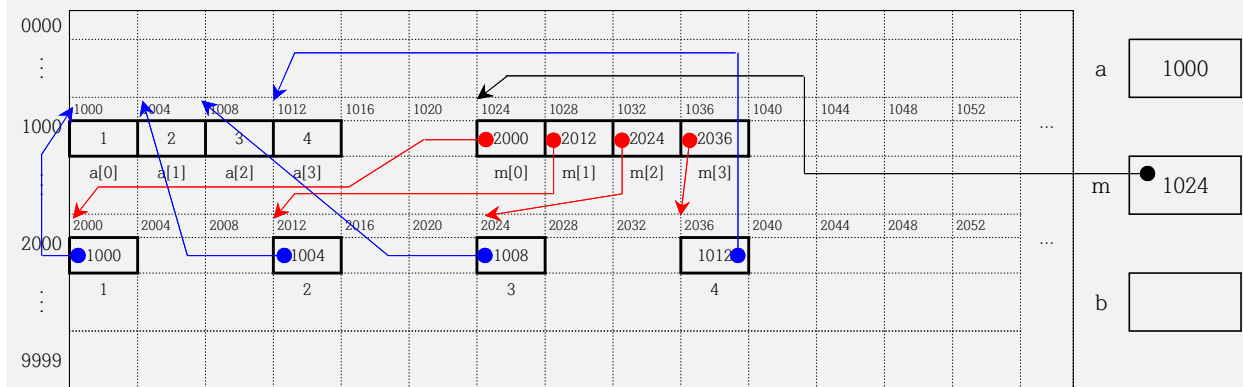
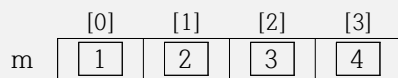
f(a) 메소드 정의부의 다음 줄인 7번째 줄부터 실행한다.

- ❶ [1, 2, 3, 4]를 인수로 f() 메소드를 호출한 후 반환받은 값을 출력한다. ❷번으로 이동한다.
- ❷ f() 메소드의 시작점이다. ❶번에서 전달한 리스트 [1, 2, 3, 4]를 매개변수 a가 받는다.



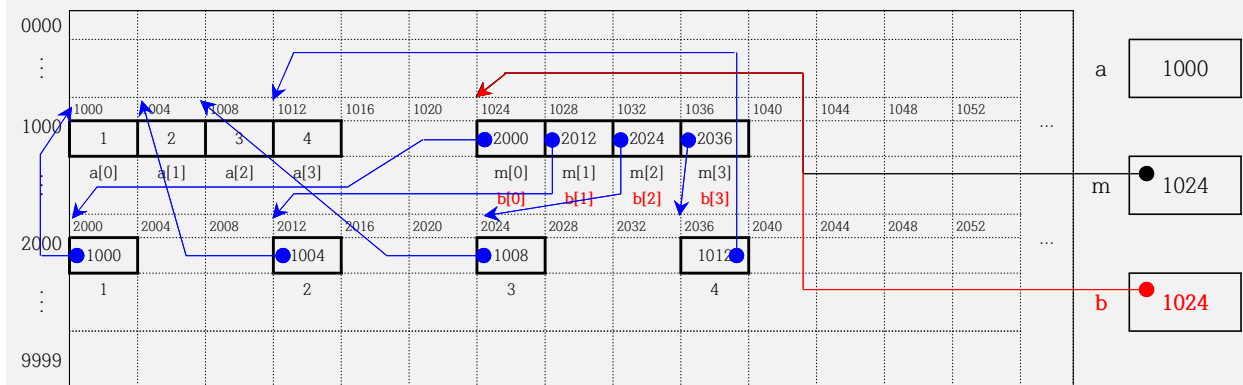
③ 컴프리헨션(Comprehension)을 사용하여 리스트 a의 각 요소를 하나씩 꺼내어 새로운 내부 리스트([x])로 감싼 2차원 리스트 m을 생성한다.

※ 컴프리헨션(Comprehension) : for 문의 반복 변수를 for 문 앞에 기술하여, 반복 변수의 값을 리스트나 딕셔너리의 형태로 반환함



④ 리스트 m을 처음부터 끝까지 추출하여 복사한 후 변수 b에 저장합니다.

※ 슬라이싱([:])을 통해 리스트 b의 틀은 새로 만들어지지만, 그 안에 담기는 요소들은 기존 m이 가리키던 내부 리스트 객체의 주소를 그대로 복사하게 됩니다. 결국 m과 b는 동일한 메모리 주소를 공유하게 되는 것입니다.



⑤ 반복 변수 i에 0부터 len(b)-1보다 작을 때까지, 즉 0부터 2까지 순차적으로 저장하면서 ⑥번을 반복 수행한다.

⑥ 'b[i+1] = b[i+1] + b[i]'와 동일하다. 현재 위치인 b[i]를 다음 위치인 b[i+1]에 덧붙여 갱신한다. 이 경우 b와 주소를 공유하는 m의 내용도 함께 갱신된다.

• 첫 번째 반복(i = 0)

-
- Figure 1: A 2D array of 1000x1000 elements. The array is divided into four quadrants. The top-left quadrant (rows 0-999, columns 0-999) contains elements labeled 'a' and 'm'. The top-right quadrant (rows 0-999, columns 1000-1999) contains elements labeled 'a' and 'm'. The bottom-left quadrant (rows 1000-1999, columns 0-999) contains elements labeled 'b' and 'm'. The bottom-right quadrant (rows 1000-1999, columns 1000-1999) contains elements labeled 'b' and 'm'. The array is divided into four quadrants by a vertical line at column 1000 and a horizontal line at row 1000. The top-left quadrant (rows 0-999, columns 0-999) contains elements labeled 'a' and 'm'. The top-right quadrant (rows 0-999, columns 1000-1999) contains elements labeled 'a' and 'm'. The bottom-left quadrant (rows 1000-1999, columns 0-999) contains elements labeled 'b' and 'm'. The bottom-right quadrant (rows 1000-1999, columns 1000-1999) contains elements labeled 'b' and 'm'.

- [illegible]

- [illegible]

- 25 -

- m[2]의 주소인 2024가 가리키는 곳의 내부 리스트는 [3, 2, 1]이므로 길이는 3
 - m[3]의 주소인 2036이 가리키는 곳의 내부 리스트는 [4, 3, 2, 1]이므로 길이는 4
- ∴ 반환되는 값은 1+2+3+4=10입니다.

⑧ ⑦번에서 돌려받은 값 10을 출력한다.

결과 10